Reflection

Working on this project gave me a much better understanding of real-time embedded systems. Before starting, I understood the concepts of interrupts, tasks, and scheduling from lectures, but implementing them on an ESP32 showed me how important timing and system design are in practice. Building an emergency stop simulation for a theme park ride helped connect my classroom concepts to a real situation where response time and reliability are critical.

If I were to complete this project again, I would spend more time designing the architecture before writing code. Initially I focused on getting individual features working first and then integrating them together. While this approach eventually worked, it resulted in additional debugging because changing one part of the application often affected another. Creating a more detailed design beforehand, including task priorities, synchronization methods, and data flow diagrams, would have made development smoother and reduced the amount of troubleshooting required later. I would also add more automated performance testing instead of manually collecting latency measurements so that comparisons between different implementations could be performed more quickly.

The most difficult part of the project was understanding how different synchronization primitives affected system behavior under different loads. Measuring interrupt latency accurately required careful placement of timestamps, ensuring the ISR remained as short as possible, and interpreting why latency changed when background tasks were enabled. Debugging timing-related issues was also more challenging than expected because problems were not always obvious from the source code alone. Small implementation details, such as where portYIELD_FROM_ISR() was called or how task priorities were assigned, had a significant impact on responsiveness.

The most valuable lesson I learned was the importance of designing software around timing rather than simply making it function correctly. In a real-time system, completing a task eventually is often not good. It must complete within a guaranteed amount of time. This project demonstrated why keeping interrupt service routines short, assigning priorities based on system criticality, and measuring worst-case execution time are essential engineering practices. It also reinforced the value of validating assumptions with real measurements instead of relying solely on theoretical expectations.

Overall, this project strengthened both my embedded programming skills and my understanding of real-time operating systems. I gained experience designing, analyzing, and defending a system based on measurable performance and engineering decisions.